PROJECT AND TEAM INFORMATION

Project Title

Code Shield : Source Code Similarity and Plagiarism Detection System

Student / Team Information

Team Name: PlagX *Team #: DSCPP-III-2026-T353*	*PlagX* *DSCPP-III-2026-T353*
Team member 1 (Team Lead) *DUBEY, GAURAV: 2510370051: 2510370051@geu.ac.in*	*DUBEY, GAURAV –* *2510370051* *2510370051@geu.ac.in* 
Team member 2 *JANDIAL, HARSHIT: 2510350032: 2510350032@geu.ac.in*	*JANDIAL, HARSHIT –* *2510350032* *2510350032@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

Teachers get a lot of programming assignments, and finding copied ones by hand takes ages. Plain text comparison isn't reliable either, because changing variable names, comments or spacing hides the copying while the logic stays identical. We want Code Shield to catch this in C/C++ programs. It borrows ideas from MOSS and Winnowing: clean and normalize the code, break it into small pieces, hash them, then compare fingerprints. The aim is to help teachers quickly find submissions that look too similar.

State of the Art / Current solution

Tools like MOSS and JPlag are the usual choice for this. They compare tokens, code structure and fingerprints instead of raw text, so renamed variables or edited comments don't fool them. But they can be heavy for a small college project, and they aren't easy to customize for a specific C/C++ assignment. We are aiming for something simpler built on normalization, k-grams, hashing and Winnowing.

Project Goals and Milestones

Our main goal is a C/C++ similarity detector that can compare many submissions and highlight the suspicious regions. These are the initial milestones:

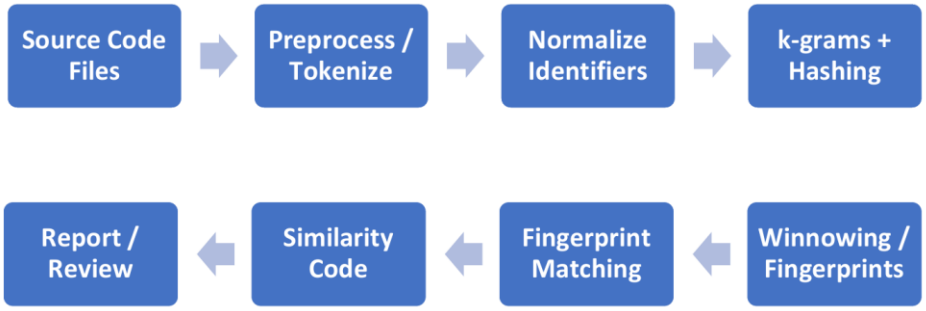
1. Read and collect source-code files.
2. Remove comments and unnecessary formatting from the code.
3. Tokenize the source code into relevant tokens.
4. Normalize identifiers so that simple variable renaming does not completely hide similarity.
5. Generate k-grams from the normalized token sequence.
6. Calculate hash values for the k-grams.
7. Apply the Winnowing algorithm to select important fingerprints.
8. Store fingerprints along with file and position information.
9. Compare fingerprints and calculate a similarity score.
10. Generate a simple report showing similar files and matching code locations.

We also want to deal with common code handed out by the instructor, so shared starter code doesn't push the similarity up for no reason.

Project Approach

We'll build it in C/C++. The first step is reading the submitted files and removing comments and extra spaces. Then we split the code into tokens like keywords, variables and operators, and normalize variable names so renaming them won't easily dodge detection. The tokens are grouped into k-grams, each one is hashed, and Winnowing picks the fingerprints we keep. Those are stored with their file and position. In the end we compare fingerprints across student files to find similar code, calculate a score, and show which files match and where.

System Architecture



Project Outcome / Deliverables

We expect to end up with a working Code Shield that compares C/C++ programs and finds similar code across student submissions. It will ignore differences in comments, spaces and variable names, use tokenization, k-grams, hashing and Winnowing to compare the important parts, and show the similarity percentage with the matching sections highlighted. It will also produce a short report so teachers can see which submissions need a second look.

Assumptions

We're assuming the work is mostly on C/C++ code and that submitted files are readable source files. Comments, extra spaces and similar differences are removed before comparing. Code the teacher hands out can be kept separate so it doesn't affect the result. The similarity percentage only says two programs look alike, and the teacher decides whether it is plagiarism.

References

1. S. Schleimer, D. S. Wilkerson and A. Aiken, "Winnowing: Local Algorithms for Document Fingerprinting," ACM SIGMOD, 2003.
2. Stanford University, "MOSS: A System for Detecting Software Similarity."
3. Stanford University, Alex Aiken Publications, "Winnowing: Local Algorithms for Document Fingerprinting."